Coach AI API - Website Developer Integration Guide

For: Frontend/Website Developers

Last Updated: February 5, 2026

API Version: 1.0.0

📖 Overview

What is Coach AI?

Coach AI analyzes coaching session videos/audio and generates professional evaluation reports based on EMCC (European Mentoring and Coaching Council) standards.

How It Works

```
User uploads video → Transcription (15-30 min) → AI Analysis → Evaluation Report
```

What You'll Build

A web interface where users can:

1. Upload coaching session videos
2. Monitor transcription progress
3. View and download evaluation reports

🚀 Getting Started

Prerequisites

- Basic knowledge of JavaScript/TypeScript
- Familiarity with REST APIs
- React, Vue, or vanilla JavaScript skills

API Endpoint

You'll receive the API URL from the backend team. It will look like:

```
https://api.coachai.com/
or
https://abc123.execute-api.eu-central-1.amazonaws.com/prod/
```

Quick Test

Test if the API is accessible:

```
curl https://YOUR-API-URL/
```

Expected response:

```
{
  "status": "healthy",
  "service": "Coach AI API",
  "version": "1.0.0"
}
```

🔐 Authentication

Current Setup

The API currently doesn't require authentication for simplicity. In production, you may need to add API keys.

Future: API Key Authentication

If API keys are added, include them in requests:

```
fetch('https://api-url/endpoint', {
  headers: {
    'Authorization': 'Bearer YOUR_API_KEY'
  }
})
```

🎯 Integration Options

Option 1: Use Our Pre-Built SDK (Recommended)

We provide a TypeScript/JavaScript SDK that handles all API calls.

Pros:

- Ready to use
- Type safety (TypeScript)
- Built-in error handling
- Progress tracking

Cons:

- Additional dependency

Option 2: Use Our React Component (Fastest)

Drop-in React component with complete UI.

Pros:

- Complete UI included
- Zero API knowledge needed
- Customizable styling

Cons:

- React only
- Less flexible

Option 3: Direct API Calls

Make HTTP requests directly to the API.

Pros:

- Framework agnostic
- Full control
- No dependencies

Cons:

- More code to write
- Manual error handling

📝 Step-by-Step Implementation

Step 1: Choose Your Approach

For React Projects: Use the React component (easiest)

For TypeScript Projects: Use the SDK

For Other Frameworks: Use direct API calls

Step 2A: Using the React Component (Recommended for React)

Installation

4. Copy `client-sdk.ts` and `react-example.tsx` to your project
5. Import the component:

```
import CoachAIUploader from './components/CoachAIUploader';

function App() {
  return (
    <div className="app">
      <h1>Coach AI</h1>
      <CoachAIUploader />
    </div>
  );
}

export default App;
```

Customization

The component accepts props for customization:

```
<CoachAIUploader
  apiUrl="https://your-api-url.com"
  maxFileSize={5 * 1024 * 1024 * 1024} // 5GB
  acceptedFormats={['video/*', 'audio/*']}
  onComplete={(report) => {
    console.log('Report ready:', report);
    // Send to your backend, show notification, etc.
  }}
/>
```

That's it! The component handles everything else.

Step 2B: Using the SDK

Installation

6. Copy `client-sdk.ts` to your project (e.g., `src/lib/coach-ai-client.ts`)
7. Install dependencies if needed:

```
npm install
# No additional dependencies needed - uses native fetch API
```

Basic Usage

```
import CoachAIClient from './lib/coach-ai-client';

// Initialize client
const client = new CoachAIClient('https://your-api-url.com');

// Get file from input
const fileInput = document.getElementById('video-upload') as
HTMLInputElement;
const file = fileInput.files[0];

// Process video with progress tracking
try {
  const result = await client.processCompleteWorkflow(file, {
    chunkSeconds: 900,  // 15 minutes per chunk
    modelSize: 'base',  // Whisper model size

    // Progress callbacks
    onUploadComplete: (upload) => {
      console.log('✅ Uploaded:', upload.key);
      showNotification('File uploaded successfully');
    },

    onTranscriptionStart: (transcription) => {
      console.log('🎬 Transcription started:', transcription.job_id);
      showProgress(20);
    },

    onTranscriptionProgress: (status) => {
      console.log('⏳ Status:', status.status);
      if (status.status === 'InProgress') showProgress(50);
```

```
      if (status.status === 'Completed') showProgress(70);
    },

    onTranscriptionComplete: (transcript) => {
      console.log('📝 Transcript ready:', transcript.word_count,
'words');
      showProgress(80);
    },

    onEvaluationComplete: (evaluation) => {
      console.log('✨ Evaluation complete!');
      showProgress(100);
      displayReport(evaluation.report);
    }
  });

  console.log('All done!', result);

} catch (error) {
  console.error('Error:', error.message);
  showError('Processing failed: ' + error.message);
}
```

Individual API Methods

You can also use individual methods for more control:

```
// 1. Upload file
const upload = await client.uploadFile(file);
console.log('File uploaded to:', upload.key);

// 2. Start transcription
const transcription = await client.startTranscription({
  video_key: upload.key,
  chunk_seconds: 900,
  model_size: 'base'
});
console.log('Job ID:', transcription.job_id);

// 3. Wait for completion
const status = await client.waitForTranscription(
  transcription.job_id,
  {
    pollInterval: 10000,  // Check every 10 seconds
    onProgress: (status) => {
      console.log('Current status:', status.status);
    }
  }
);

// 4. Get transcript
const transcript = await client.getTranscript(status.transcript_key);
console.log('Transcript:', transcript.content);

// 5. Generate evaluation
const evaluation = await client.evaluateCoaching({
```

```
  transcript_key: status.transcript_key
});
console.log('Report:', evaluation.report);
```

Step 2C: Direct API Calls (Framework Agnostic)

If you prefer not to use our SDK, here's how to make direct API calls:

1. Upload File

```
async function uploadFile(file) {
  const formData = new FormData();
  formData.append('file', file);

  const response = await fetch('https://api-url/api/v1/upload', {
    method: 'POST',
    body: formData
  });

  if (!response.ok) {
    throw new Error(`Upload failed: ${response.status}`);
  }

  return await response.json();
  // Returns: { key: "uploads/...", url: "...", size: 123456 }
}
```

2. Start Transcription

```
async function startTranscription(videoKey) {
  const response = await fetch('https://api-url/api/v1/transcribe', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      video_key: videoKey,
      chunk_seconds: 900,
      model_size: 'base'
    })
  });

  if (!response.ok) {
    throw new Error(`Transcription failed: ${response.status}`);
  }

  return await response.json();
  // Returns: { job_id: "...", status: "InProgress", ... }
}
```

3. Check Status (with Polling)

```
async function waitForTranscription(jobId) {
  const maxAttempts = 180; // 30 minutes max
  const pollInterval = 10000; // 10 seconds
```

```
  for (let i = 0; i < maxAttempts; i++) {
    const response = await fetch(
      `https://api-url/api/v1/transcribe/${jobId}`
    );

    const status = await response.json();
    console.log('Status:', status.status);

    if (status.status === 'Completed') {
      return status;
    }

    if (status.status === 'Failed') {
      throw new Error(status.error || 'Transcription failed');
    }

    // Wait before next check
    await new Promise(resolve => setTimeout(resolve, pollInterval));
  }

  throw new Error('Transcription timeout');
}
```

4. Get Transcript

```
async function getTranscript(transcriptKey) {
  const response = await fetch(
    `https://api-url/api/v1/transcript/${transcriptKey}`
  );

  if (!response.ok) {
    throw new Error(`Failed to get transcript: ${response.status}`);
  }

  return await response.json();
  // Returns: { key: "...", content: "...", length: 12345, word_count:
1500 }
}
```

5. Generate Evaluation

```
async function generateEvaluation(transcriptKey) {
  const response = await fetch('https://api-url/api/v1/evaluate', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      transcript_key: transcriptKey
    })
  });

  if (!response.ok) {
    throw new Error(`Evaluation failed: ${response.status}`);
  }
```

```
  return await response.json();
  // Returns: { evaluation_id: "...", report: "...", timestamp: "..." }
}
```

Complete Workflow Example

```
async function processVideo(file) {
  try {
    // 1. Upload
    updateUI('Uploading file...');
    const upload = await uploadFile(file);
    console.log('Uploaded:', upload.key);

    // 2. Start transcription
    updateUI('Starting transcription...');
    const transcription = await startTranscription(upload.key);
    console.log('Job started:', transcription.job_id);

    // 3. Wait for completion
    updateUI('Transcribing (this may take 15-30 minutes)...');
    const status = await waitForTranscription(transcription.job_id);
    console.log('Transcription complete!');

    // 4. Get transcript (optional)
    const transcript = await getTranscript(status.transcript_key);
    console.log('Transcript:', transcript.word_count, 'words');

    // 5. Generate evaluation
    updateUI('Generating evaluation report...');
    const evaluation = await generateEvaluation(status.transcript_key);
    console.log('Evaluation complete!');

    // 6. Display report
    displayReport(evaluation.report);
    updateUI('Complete!');

  } catch (error) {
    console.error('Error:', error);
    updateUI('Error: ' + error.message);
  }
}

// Usage
const fileInput = document.getElementById('video-upload');
fileInput.addEventListener('change', (e) => {
  const file = e.target.files[0];
  if (file) {
    processVideo(file);
  }
});
```

💻 Code Examples

Example 1: Vanilla JavaScript (Simple Form)

```
<!DOCTYPE html>
<html>
<head>
  <title>Coach AI</title>
  <style>
    .container { max-width: 800px; margin: 50px auto; padding: 20px; }
    .upload-box { border: 2px dashed #ccc; padding: 40px; text-align:
center; }
    .progress-bar { width: 100%; height: 30px; background: #f0f0f0;
border-radius: 15px; overflow: hidden; margin: 20px 0; }
    .progress-fill { height: 100%; background: linear-gradient(90deg,
#4CAF50, #45a049); transition: width 0.3s; }
    .report { background: #f9f9f9; padding: 20px; border-radius: 8px;
white-space: pre-wrap; }
    .hidden { display: none; }
  </style>
</head>
<body>
  <div class="container">
    <h1>Coach AI - Video Analysis</h1>

    <!-- Upload Form -->
    <div id="upload-section">
      <div class="upload-box">
        <input type="file" id="video-input" accept="video/*,audio/*" />
        <p>Select a coaching session video to analyze</p>
        <button id="upload-btn" onclick="processVideo()">Upload &
Analyze</button>
      </div>
    </div>

    <!-- Progress Section -->
    <div id="progress-section" class="hidden">
      <h3>Processing...</h3>
      <div class="progress-bar">
        <div id="progress-fill" class="progress-fill" style="width:
0%"></div>
      </div>
      <p id="status-message">Initializing...</p>
    </div>

    <!-- Results Section -->
    <div id="results-section" class="hidden">
      <h3>Coaching Evaluation Report</h3>
      <div id="report" class="report"></div>
      <button onclick="downloadReport()">Download Report</button>
      <button onclick="reset()">Analyze Another Video</button>
    </div>
  </div>

  <script>
```

```
    const API_URL = 'https://your-api-url.com';
    let currentReport = '';

    function updateProgress(percent, message) {
      document.getElementById('progress-fill').style.width = percent +
'%';
      document.getElementById('status-message').textContent = message;
    }

    function showSection(section) {
      document.getElementById('upload-section').classList.add('hidden');

document.getElementById('progress-section').classList.add('hidden');
      document.getElementById('results-section').classList.add('hidden');
      document.getElementById(section).classList.remove('hidden');
    }

    async function processVideo() {
      const fileInput = document.getElementById('video-input');
      const file = fileInput.files[0];

      if (!file) {
        alert('Please select a file');
        return;
      }

      showSection('progress-section');

      try {
        // Upload
        updateProgress(10, 'Uploading file...');
        const formData = new FormData();
        formData.append('file', file);

        const uploadResponse = await fetch(`${API_URL}/api/v1/upload`, {
          method: 'POST',
          body: formData
        });
        const upload = await uploadResponse.json();

        // Start transcription
        updateProgress(20, 'Starting transcription...');
        const transcribeResponse = await
fetch(`${API_URL}/api/v1/transcribe`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
            video_key: upload.key,
            chunk_seconds: 900,
            model_size: 'base'
          })
        });
        const transcription = await transcribeResponse.json();

        // Wait for completion
```

```
        updateProgress(30, 'Transcribing (15-30 minutes)...');
        const status = await pollJobStatus(transcription.job_id);

        // Generate evaluation
        updateProgress(80, 'Generating evaluation report...');
        const evalResponse = await fetch(`${API_URL}/api/v1/evaluate`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
            transcript_key: status.transcript_key
          })
        });
        const evaluation = await evalResponse.json();

        // Show results
        updateProgress(100, 'Complete!');
        currentReport = evaluation.report;
        document.getElementById('report').textContent =
evaluation.report;
        showSection('results-section');

      } catch (error) {
        alert('Error: ' + error.message);
        showSection('upload-section');
      }
    }

    async function pollJobStatus(jobId) {
      const maxAttempts = 180;
      const pollInterval = 10000;

      for (let i = 0; i < maxAttempts; i++) {
        const response = await
fetch(`${API_URL}/api/v1/transcribe/${jobId}`);
        const status = await response.json();

        const progress = 30 + (i / maxAttempts * 50); // 30% to 80%
        updateProgress(progress, `Transcribing: ${status.status}`);

        if (status.status === 'Completed') {
          return status;
        }

        if (status.status === 'Failed') {
          throw new Error(status.error || 'Transcription failed');
        }

        await new Promise(resolve => setTimeout(resolve, pollInterval));
      }

      throw new Error('Transcription timeout');
    }

    function downloadReport() {
      const blob = new Blob([currentReport], { type: 'text/plain' });
```

```
      const url = URL.createObjectURL(blob);
      const a = document.createElement('a');
      a.href = url;
      a.download = 'coaching-evaluation-' + Date.now() + '.txt';
      a.click();
      URL.revokeObjectURL(url);
    }

    function reset() {
      document.getElementById('video-input').value = '';
      currentReport = '';
      showSection('upload-section');
    }
  </script>
</body>
</html>
```

Example 2: Vue.js Component

```
<template>
  <div class="coach-ai-uploader">
    <h2>Coach AI Video Analysis</h2>

    <!-- Upload Section -->
    <div v-if="stage === 'idle'" class="upload-section">
      <input
        type="file"
        @change="handleFileChange"
        accept="video/*,audio/*"
        ref="fileInput"
      />
      <button @click="processVideo" :disabled="!selectedFile">
        Upload & Analyze
      </button>
    </div>

    <!-- Progress Section -->
    <div v-if="stage !== 'idle' && stage !== 'complete'"
class="progress-section">
      <div class="progress-bar">
        <div class="progress-fill" :style="{ width: progress + '%'
}"></div>
      </div>
      <p>{{ statusMessage }}</p>
    </div>

    <!-- Results Section -->
    <div v-if="stage === 'complete'" class="results-section">
      <h3>Coaching Evaluation Report</h3>
      <pre class="report">{{ evaluationReport }}</pre>
      <button @click="downloadReport">Download Report</button>
      <button @click="reset">Analyze Another Video</button>
    </div>

    <!-- Error Display -->
```

```
    <div v-if="error" class="error-message">
      {{ error }}
    </div>
  </div>
</template>

<script>
export default {
  name: 'CoachAIUploader',
  props: {
    apiUrl: {
      type: String,
      required: true
    }
  },
  data() {
    return {
      selectedFile: null,
      stage: 'idle', // idle, uploading, transcribing, evaluating,
complete
      progress: 0,
      statusMessage: '',
      evaluationReport: '',
      error: null
    };
  },
  methods: {
    handleFileChange(event) {
      this.selectedFile = event.target.files[0];
      this.error = null;
    },

    async processVideo() {
      if (!this.selectedFile) return;

      this.error = null;
      this.stage = 'uploading';
      this.progress = 10;
      this.statusMessage = 'Uploading file...';

      try {
        // Upload
        const formData = new FormData();
        formData.append('file', this.selectedFile);

        const uploadResponse = await
fetch(`${this.apiUrl}/api/v1/upload`, {
          method: 'POST',
          body: formData
        });
        const upload = await uploadResponse.json();

        // Transcribe
        this.stage = 'transcribing';
        this.progress = 20;
```

```
        this.statusMessage = 'Starting transcription...';

        const transcribeResponse = await
fetch(`${this.apiUrl}/api/v1/transcribe`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
            video_key: upload.key,
            chunk_seconds: 900,
            model_size: 'base'
          })
        });
        const transcription = await transcribeResponse.json();

        // Wait for completion
        this.progress = 30;
        this.statusMessage = 'Transcribing (this may take 15-30
minutes)...';
        const status = await this.pollJobStatus(transcription.job_id);

        // Evaluate
        this.stage = 'evaluating';
        this.progress = 80;
        this.statusMessage = 'Generating evaluation report...';

        const evalResponse = await
fetch(`${this.apiUrl}/api/v1/evaluate`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
            transcript_key: status.transcript_key
          })
        });
        const evaluation = await evalResponse.json();

        // Complete
        this.stage = 'complete';
        this.progress = 100;
        this.evaluationReport = evaluation.report;

      } catch (error) {
        this.error = error.message;
        this.stage = 'idle';
      }
    },

    async pollJobStatus(jobId) {
      const maxAttempts = 180;
      const pollInterval = 10000;

      for (let i = 0; i < maxAttempts; i++) {
        const response = await
fetch(`${this.apiUrl}/api/v1/transcribe/${jobId}`);
        const status = await response.json();
```

```
        this.progress = 30 + (i / maxAttempts * 50);
        this.statusMessage = `Transcribing: ${status.status}`;

        if (status.status === 'Completed') {
          return status;
        }

        if (status.status === 'Failed') {
          throw new Error(status.error || 'Transcription failed');
        }

        await new Promise(resolve => setTimeout(resolve, pollInterval));
      }

      throw new Error('Transcription timeout');
    },

    downloadReport() {
      const blob = new Blob([this.evaluationReport], { type: 'text/plain'
});
      const url = URL.createObjectURL(blob);
      const a = document.createElement('a');
      a.href = url;
      a.download = `coaching-evaluation-${Date.now()}.txt`;
      a.click();
      URL.revokeObjectURL(url);
    },

    reset() {
      this.selectedFile = null;
      this.stage = 'idle';
      this.progress = 0;
      this.statusMessage = '';
      this.evaluationReport = '';
      this.error = null;
      this.$refs.fileInput.value = '';
    }
  }
};
</script>

<style scoped>
.coach-ai-uploader {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px;
}

.progress-bar {
  width: 100%;
  height: 30px;
  background: #f0f0f0;
  border-radius: 15px;
  overflow: hidden;
  margin: 20px 0;
```

```
}

.progress-fill {
  height: 100%;
  background: linear-gradient(90deg, #4CAF50, #45a049);
  transition: width 0.3s;
}

.report {
  background: #f9f9f9;
  padding: 20px;
  border-radius: 8px;
  white-space: pre-wrap;
  max-height: 400px;
  overflow-y: auto;
}

.error-message {
  background: #f8d7da;
  color: #721c24;
  padding: 15px;
  border-radius: 6px;
  margin: 20px 0;
}

button {
  background: #4CAF50;
  color: white;
  border: none;
  padding: 12px 24px;
  border-radius: 6px;
  cursor: pointer;
  margin: 5px;
}

button:hover {
  background: #45a049;
}

button:disabled {
  background: #ccc;
  cursor: not-allowed;
}
</style>
```

🎨 UI/UX Best Practices

1. File Upload

Good Practices:

- Accept video and audio formats: `.mov`, `.mp4`, `.mkv`, `.wav`, `.mp3`
- Show file size limit (recommend max 5GB)

- Display selected file name and size
- Add drag-and-drop support
- Show upload progress

Example:

```
<input
  type="file"
  accept="video/*,audio/*,.mov,.mp4,.mkv,.wav,.mp3"
  id="video-upload"
/>
<p>Maximum file size: 5GB</p>
```

2. Progress Indication

Transcription takes 15-30 minutes, so provide clear feedback:

```
// Show estimated time
const estimatedMinutes = Math.ceil(fileSizeInMB / 100);
updateStatus(`Transcribing... (estimated ${estimatedMinutes} minutes)`);

// Update progress regularly
setInterval(() => {
  // Check status every 10 seconds
  checkJobStatus(jobId);
}, 10000);
```

Visual Elements:

- Progress bar (0-100%)
- Stage indicators (Upload → Transcribe → Evaluate)
- Time estimates
- Current status messages

3. User Experience

During Processing:

- Allow users to leave the page (save job ID in localStorage)
- Send email notification when complete (backend feature)
- Show "Can take 15-30 minutes" warning upfront

After Completion:

- Preview first 500 characters of report
- Download as PDF or TXT
- Share via email
- Save to user account

4. Error Handling

Common Errors:

- File too large

- Invalid file format
- Transcription timeout
- Network errors

User-Friendly Messages:

```
const errorMessages = {
  'File too large': 'Your file exceeds the 5GB limit. Please try a
smaller file.',
  'Invalid format': 'Please upload a video or audio file (.mov, .mp4,
.mkv, .wav, .mp3)',
  'Transcription failed': 'We couldn\'t process your video. Please try
again or contact support.',
  'Network error': 'Connection lost. Please check your internet and try
again.'
};
```

🚨 Error Handling

HTTP Status Codes

Code	Meaning	Action
200	Success	Continue
400	Bad Request	Check request parameters
404	Not Found	Job/file doesn't exist
500	Server Error	Retry or contact support
503	Service Unavailable	Server down, retry later

Example Error Handler

```
async function handleAPICall(apiFunction) {
  try {
    return await apiFunction();
  } catch (error) {
    if (error.message.includes('404')) {
      return { error: 'Resource not found' };
    } else if (error.message.includes('500')) {
      return { error: 'Server error. Please try again.' };
    } else if (error.message.includes('timeout')) {
      return { error: 'Request timeout. Please check your connection.' };
    } else {
      return { error: 'An unexpected error occurred: ' + error.message };
    }
  }
}

// Usage
const result = await handleAPICall(() => uploadFile(file));
if (result.error) {
  showErrorToUser(result.error);
}
```

Retry Logic

```
async function fetchWithRetry(url, options, maxRetries = 3) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      const response = await fetch(url, options);
      if (response.ok) return response;

      // Don't retry on 4xx errors
      if (response.status >= 400 && response.status < 500) {
        throw new Error(`HTTP ${response.status}`);
      }

      // Retry on 5xx errors
      if (i < maxRetries - 1) {
        await new Promise(resolve => setTimeout(resolve, 1000 * (i +
1)));
      }
    } catch (error) {
      if (i === maxRetries - 1) throw error;
    }
  }
}
```

🧪 Testing

1. Test with Sample Video

Use a short test video (1-2 minutes) for faster testing:

```
// Test mode - use smaller chunks for faster processing
const testMode = true;

const request = {
  video_key: upload.key,
  chunk_seconds: testMode ? 60 : 900,  // 1 min chunks in test mode
  model_size: testMode ? 'tiny' : 'base'  // Faster model in test mode
};
```

2. Mock API Responses (Development)

```
// Mock mode for UI development
const MOCK_MODE = false;

async function uploadFile(file) {
  if (MOCK_MODE) {
    await delay(1000);
    return {
      key: 'mock/test-video.mov',
      url: 'https://mock-url',
      size: file.size
    };
  }
```

```
  // Real API call
  return realUploadFile(file);
}

async function pollJobStatus(jobId) {
  if (MOCK_MODE) {
    await delay(2000);
    return {
      job_id: jobId,
      status: 'Completed',
      transcript_key: 'mock/transcript.txt'
    };
  }

  // Real polling
  return realPollJobStatus(jobId);
}

function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
```

3. Test Checklist

- [ ] File upload works for different formats
- [ ] Progress updates correctly
- [ ] Handles large files (test with 2GB+ file)
- [ ] Error messages display properly
- [ ] Download report works
- [ ] Page refresh during processing (recover job ID)
- [ ] Multiple files in sequence
- [ ] Mobile responsiveness
- [ ] Network disconnection handling

✅ Production Checklist

Before going live:

Frontend

- [ ] API URL configured (not hardcoded localhost)
- [ ] Error messages are user-friendly
- [ ] Loading states for all async operations
- [ ] File size validation (max 5GB)
- [ ] File type validation
- [ ] Progress updates every 10 seconds
- [ ] Handle page refresh (save job ID)
- [ ] Mobile responsive design
- [ ] Accessibility (ARIA labels, keyboard navigation)
- [ ] Analytics tracking (upload, completion, errors)

Backend Integration

- [ ] CORS configured for your domain
- [ ] API endpoint is HTTPS
- [ ] Rate limiting understood
- [ ] Error logging in place
- [ ] Monitoring/alerts set up

User Experience

- [ ] Clear instructions for users
- [ ] Expected processing time displayed (15-30 min)
- [ ] Email notification option (if available)
- [ ] FAQ or help section
- [ ] Support contact information
- [ ] Privacy policy (video handling)

Performance

- [ ] Compress images/assets
- [ ] Lazy load heavy components
- [ ] CDN for static assets
- [ ] Browser caching configured
- [ ] Test on slow connections

📞 Support & Questions

Common Questions

Q: How long does processing take?

A: Typically 10-15 minutes depending on video length. A 1-hour video takes about 20 minutes.

Q: What's the maximum file size?

A: 5GB recommended. Larger files may timeout.

Q: Can users leave the page during processing?

A: Yes, save the job ID and allow them to check status later.

Q: Is the video stored permanently?

A: Videos are stored in S3. Configure lifecycle policies as needed.

Q: Can I customize the evaluation criteria?

A: Yes, pass the emcc_rules parameter in the evaluate request.

Getting Help

8. Check the interactive API docs at `/docs`
9. Review this guide
10. Check CloudWatch logs for backend errors
11. Contact the backend team with:

- Job ID
- Error message
- Steps to reproduce